

Optimización Bayesiana Multimodelo con Optuna: Espacios de Sintonización y Dinámicas de Aprendizaje en Arquitectura Secuencial y Tabular

El ajuste sistemático de hiperparámetros representa un pilar crítico en la ingeniería de modelos de aprendizaje automático, garantizando que un algoritmo alcance su límite superior de rendimiento predictivo y robustez generalizadora¹. Históricamente, las búsquedas exhaustivas en cuadrícula (Grid Search) y los métodos estocásticos sin memoria (Random Search) han dominado el despliegue industrial; sin embargo, adolecen de ineficiencias severas debido a la falta de adaptabilidad y el alto coste computacional en espacios de búsqueda de alta dimensionalidad¹. La optimización bayesiana, en particular el estimador de Parzen estructurado en árbol (TPE, *Tree-structured Parzen Estimator*), mitiga estas limitaciones mediante la formulación probabilística del rendimiento del modelo a partir del historial de evaluaciones previas, guiando la exploración de manera inteligente hacia regiones de alta densidad de rendimiento¹.

Al enfrentarse a una colección diversa de arquitecturas (desde estimadores tabulares no lineales clásicos hasta modernos Transformers secuenciales y variantes de memoria extendida), el diseñador del sistema debe configurar espacios de sintonización que respeten tanto las restricciones matemáticas de cada arquitectura como los límites computacionales de la infraestructura disponible⁵. La presente investigación detalla la formulación técnica de los espacios de búsqueda optimizados para implementar en Optuna, analizando la dinámica de sintonización y las interdependencias de parámetros de cada una de las ocho familias de modelos seleccionadas.

Modelos Clásicos de Base No Lineal y Redes Recurrentes de Compuerta

Random Forest

El modelo de bosque aleatorio es un ensamble de árboles de decisión independientes entrenados mediante agregación por arranque (*bagging*) y selección aleatoria de subespacios de características¹. Aunque los métodos de aumento de gradiente (*gradient boosting*) a menudo superan ligeramente a los bosques aleatorios en precisión pura, estos últimos exhiben una estabilidad matemática excepcional derivada de la ley de los grandes números, reduciendo la varianza del modelo sin alterar de manera significativa el sesgo global³.

Durante la optimización con Optuna, el hiperparámetro `n_estimators` plantea un desafío particular para el estimador TPE tradicional: la precisión suele mejorar de forma monótona con el tamaño del ensamble, lo que empuja al optimizador a seleccionar valores cada vez más altos,

incrementando linealmente el tiempo de cómputo sin ofrecer mejoras estadísticas sustanciales⁴. Para contrarrestar esta ineficiencia, es posible emplear la puntuación fuera de bolsa (OOB, *Out-of-Bag*) como un estimador de validación alternativo en una sola corrida de entrenamiento, evitando la sobrecarga de la validación cruzada y permitiendo detener la exploración de tamaño mediante algoritmos adaptativos de búsqueda de meseta (plateau search)⁴.

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
n_estimators	suggest_int	[50, 1000] (Paso: 50)	Define el volumen de estimadores débiles. Controla la varianza; la sintonización debe limitarse para evitar costes computacionales redundantes ¹ .
max_depth	suggest_categorical	[None, 5, 10, 20, 30, 50]	Determina el límite de particiones verticales. Evita el sobreajuste aislando la modelización de ruido en niveles profundos ¹ .
min_samples_split	suggest_int	[2, 20]	Cantidad mínima de instancias requeridas para subdividir un nodo interno de un árbol ¹ .
min_samples_leaf	suggest_int	[1, 20]	Umbral mínimo de datos permitidos en un nodo terminal, suavizando las fronteras de decisión frente a

			valores atípicos.
max_features	suggest_categorica 	["sqrt", "log2", None]	Cardinalidad de la subsección de variables analizadas en cada nodo. None evalúa todas las variables ⁴ .
bootstrap	suggest_categorica 	[True, False]	Determina si el subconjunto de entrenamiento de cada estimador se extrae con reemplazo del conjunto de datos original ⁴ .
criterion	suggest_categorica 	["gini", "entropy"]	Métrica matemática para evaluar la pureza y ganancia de información en las divisiones locales ¹ .

LSTM (Long Short-Term Memory)

La red de memoria a largo corto plazo resuelve el problema del desvanecimiento y la explosión del gradiente en dependencias de largo alcance mediante un canal de flujo de información lineal controlado por compuertas matemáticas⁸. La transición de estado se rige por un estado

de celda c_t y un estado oculto h_t , actualizados mediante compuertas de olvido (f_t), entrada (i_t) y salida (o_t)¹¹:

$$\begin{aligned}
 f_t &= \sigma(W_f x_t + R_f h_{t-1} + b_f) \\
 i_t &= \sigma(W_i x_t + R_i h_{t-1} + b_i) \\
 o_t &= \sigma(W_o x_t + R_o h_{t-1} + b_o) \\
 c_t &= f_t \odot c_{t-1} + i_t \odot \tanh(W_c x_t + R_c h_{t-1} + b_c) \\
 h_t &= o_t \odot \tanh(c_t)
 \end{aligned}$$

El ajuste óptimo de una arquitectura LSTM requiere equilibrar la expresividad representacional, dictada por el tamaño de la dimensión oculta (`hidden_size`), con medidas de regularización estricta como el descarte aleatorio (*dropout*)¹³. En tareas complejas de series temporales, el uso de optimizadores avanzados (como AdamW o Nadam) y la calibración fina de la tasa de aprendizaje resultan determinantes para evitar mínimos locales y divergencias numéricas¹⁴.

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
<code>hidden_size</code>	<code>suggest_int</code>	[16, 512] (Paso: 16)	Capacidad dimensional de la memoria interna y estado oculto de la red ¹⁶ .
<code>num_layers</code>	<code>suggest_int</code>	[1, 4]	Cantidad de capas apiladas para la abstracción jerárquica de características temporales ¹⁴ .
<code>dropout</code>	<code>suggest_float</code>	[0.0, 0.5]	Fracción de conexiones anuladas aleatoriamente entre capas para prevenir co-adaptación de pesos ¹³ .

learning_rate	suggest_float	[1e-5, 1e-2] (Log)	Controla el tamaño de actualización de pesos en la dirección opuesta al gradiente ¹³ .
batch_size	suggest_categorica l	[16, 32, 64, 128, 256]	Tamaño del lote de datos procesados por iteración de retropropagación ¹³ .
optimizer	suggest_categorica l	["Adam", "AdamW", "RMSprop", "SGD"]	Algoritmo optimizador. AdamW desacopla el decaimiento de pesos de la actualización basada en momentos ¹⁴ .

GRU (Gated Recurrent Unit)

La unidad recurrente de compuertas condensa la topología de la LSTM eliminando el canal de memoria de celda explícito y combinando los estados lógicos en un único vector continuo h_t ⁸.

Al operar con solo dos compuertas (la compuerta de actualización z_t y la compuerta de reinicio r_t), la celda GRU reduce considerablemente la cantidad de parámetros entrenables, lo que disminuye tanto la demanda de memoria como el tiempo de computación necesario por cada época de entrenamiento⁸:

$$\begin{aligned}
 z_t &= \sigma(W_z x_t + R_z h_{t-1} + b_z) \\
 r_t &= \sigma(W_r x_t + R_r h_{t-1} + b_r) \\
 \tilde{h}_t &= \tanh(W_h x_t + r_t \odot (R_h h_{t-1}) + b_h)
 \end{aligned}$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

A pesar de su ligereza, GRU puede presentar limitaciones para retener patrones complejos de muy largo plazo en comparación con una LSTM en conjuntos de datos masivos⁹. En Optuna, la

sintonización de GRU debe centrarse en buscar configuraciones de regularización L_2 (weight_decay) más restrictivas y tamaños de lotes estables para explotar su velocidad de convergencia sin caer en sobreajuste local⁹.

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
hidden_size	suggest_int	[16, 512] (Paso: 16)	Dimensión interna del vector de estados recurrentes unificado ⁹ .
num_layers	suggest_int	[1, 4]	Número de capas GRU apiladas para la extracción de patrones de nivel superior.
dropout	suggest_float	[0.0, 0.5]	Tasa de regularización por descarte aleatorio en las conexiones de las capas.
learning_rate	suggest_float	[1e-5, 1e-2] (Log)	Tasa de aprendizaje empleada para la corrección estocástica de los pesos de la red ¹⁶ .
weight_decay	suggest_float	[1e-6, 1e-2] (Log)	Parámetro de penalización de norma L_2 para mitigar la inestabilidad de pesos.

batch_size	suggest_categorica l	[16, 32, 64, 128, 256]	Volumen del lote que influye directamente en la varianza estocástica de los gradientes calculados.
------------	-------------------------	------------------------	--

Arquitecturas Convolucionales y Modelos Híbridos Secuenciales

CNN-LSTM

La arquitectura híbrida CNN-LSTM se basa en una jerarquía espacial y temporal donde una serie de capas convolucionales unidimensionales (1D-CNN) operan en primera instancia sobre las series de tiempo crudas⁷. Estas capas convolucionales actúan como extractores automáticos de características locales no lineales (como picos de frecuencia u oscilaciones específicas), proyectando mapas de características simplificados de menor dimensionalidad espacial hacia las capas LSTM subsiguientes, las cuales se encargan de modelar las relaciones de largo alcance y la secuencia dinámica global⁷.

La calibración de este modelo en Optuna exige la estructuración de dependencias cruzadas complejas². Los filtros convolucionales (conv_filters) y el tamaño de kernel (kernel_size) deben ajustarse de manera simultánea con las dimensiones de la capa LSTM². Un tamaño de kernel mal seleccionado puede ignorar patrones locales transitorios de alta relevancia física, mientras que una subcapa convolucional sobreparametrizada sobrecargará de ruido el procesamiento de la etapa recurrente².

Secuencia de Entrada -> [Conv 1D] -> [Activation/BatchNorm] -> [MaxPooling] -> [LSTM Layer] -> [Dropout] -> [Linear Out]

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
-----------	-----------------------------	---------------------------------	---

conv_filters	suggest_int	[16, 128] (Paso: 16)	Número de canales o filtros convolucionales aplicados sobre la dimensión secuencial ² .
kernel_size	suggest_int	[2, 8]	Dimensión espacial de la ventana deslizante que captura interacciones locales continuas ¹⁸ .
pool_size	suggest_int	[2, 4]	Factor de submuestreo máximo. Disminuye la longitud de la serie para reducir la carga de la LSTM.
lstm_hidden_size	suggest_int	[32, 256] (Paso: 32)	Dimensión del espacio de estados internos de la capa secuencial profunda ¹³ .
lstm_layers	suggest_int	[1, 3]	Cantidad de capas recurrentes de tipo LSTM apiladas tras la fase convolucional.
dropout	suggest_float	[0.1, 0.5]	Probabilidad de descarte en el bloque totalmente conectado posterior para evitar el sobreajuste ² .

learning_rate	suggest_float	[1e-5, 1e-2] (Log)	Tasa de aprendizaje que regula la convergencia coordinada del codificador CNN y el decodificador LSTM ² .
---------------	---------------	--------------------	--

TCN (Temporal Convolutional Network)

Las redes convolucionales temporales (TCN) reemplazan la naturaleza secuencial recurrente por operaciones de convolución causal y dilatada con conexiones de salto residual²⁰. Las convoluciones causales garantizan que la predicción en el instante t dependa únicamente de las entradas correspondientes a índices temporales menores o iguales a t , eliminando de raíz la fuga de información del futuro²⁰. Las convoluciones dilatadas introducen un paso de muestreo d en el kernel convolucional, permitiendo expandir el campo receptivo de la red de manera exponencial con la profundidad sin perder resolución de la serie por submuestreo²⁰.

El campo receptivo teórico (RF) para una red TCN con un tamaño de kernel K y un número de bloques residuales L donde la dilatación de la capa l crece como $d_l = 2^l$ se modela mediante la siguiente expresión matemática²²:

$$RF = 1 + \sum_{l=0}^{L-1} (K - 1) \cdot 2^l = 1 + (K - 1) \cdot (2^L - 1)$$

Para redes extremadamente profundas en series temporales de alta resolución, la dilatación exponencial puede crecer hasta generar desajustes de memoria por un acolchado (*padding*) excesivo²². Se introduce el hiperparámetro `dilation_reset` para reiniciar periódicamente el factor de dilatación a la unidad en etapas avanzadas de la arquitectura, estabilizando los requerimientos de memoria y manteniendo un campo receptivo controlado²².

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
-----------	-----------------------------	---------------------------------	---

depth	suggest_int	[2, 6]	Número total de bloques de convolución residual dilatada en el grafo de la red ²³ .
num_channels	suggest_categorica l	[[16,32,64], [32,64,128], [64,128,256]]	Lista que especifica los canales internos por cada bloque residual sucesivo de la TCN ¹⁸ .
kernel_size	suggest_int	[2, 8]	Dimensión del núcleo causal de la convolución 1D. Afecta directamente al campo receptivo de la red ¹⁸ .
dilation_reset	suggest_categorica l	[None, 8, 16, 32]	Umbral donde se fuerza el reinicio secuencial de la dilatación a valores iniciales de base ²² .
dropout	suggest_float	[0.0, 0.5]	Probabilidad de desactivación de neuronas aplicada internamente en las capas del bloque residual ¹⁸ .
use_norm	suggest_categorica l	["weight_norm", "batch_norm", "none"]	Mecanismo de regularización interna de activaciones. La normalización de pesos preserva la estabilidad del gradiente ¹⁸ .

use_skip_connections	suggest_categorical	[True, False]	Determina el uso de conexiones directas desde la salida de cada bloque residual hacia la salida de la red ²² .
----------------------	---------------------	---------------	---

Modelos Basados en Autoatención y Patrones Secuenciales de Vanguardia

Time Series Transformer

El traslado del modelo Transformer estándar a series temporales continuas requiere reemplazar la capa de embeddings discretos por una capa de proyección lineal que mapea valores numéricos unidimensionales o multivariados continuos al espacio vectorial interno del modelo d_model ²⁷. A este embedding se le suma una codificación posicional (ya sea mediante funciones senoidales estáticas o vectores de pesos aprendidos) para registrar el orden cronológico de la serie²⁷.

En la optimización con Optuna de este tipo de Transformer, la inicialización de la capa de atención de cabezales múltiples impone una restricción estructural estricta: la dimensión interna proyectada (d_model) debe ser perfectamente divisible por el número de cabezales de atención ($nhead$)³⁰. De no cumplirse esta divisibilidad, las operaciones internas de separación de tensores para proyectar subespacios de atención paralelos fallarán en tiempo de ejecución. Para garantizar esta condición de consistencia matemática dentro del espacio de muestreo bayesiano de Optuna sin penalizar la aleatoriedad de la búsqueda, se puede implementar lógica de parametrización dependiente de la siguiente forma:

Python

```
# Muestreo independiente de los cabezales
nhead = trial.suggest_categorical("trans_nhead", 2 4 8 16)
# Selección de un multiplicador entero
d_model_multiplier = trial.suggest_int("trans_d_model_multiplier", 8 32)
# d_model consistente por construcción matemática
d_model = nhead * d_model_multiplier
```

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
nhead	suggest_categorical	[2, 4, 8, 16]	Número de proyecciones independientes para la atención distribuida paralela ³⁰ .
trans_d_model_multiplier	suggest_int	[8, 32]	Factor multiplicador entero que se escala con nhead para determinar el tamaño exacto de d_model.
num_layers	suggest_int	[1, 6]	Cantidad de bloques codificadores de atención del Transformer aplicados secuencialmente ³⁰ .
dim_feedforward	suggest_int	[64, 1024] (Paso: 64)	Dimensión interna de la capa totalmente conectada posicionada después de la autoatención ³¹ .
dropout	suggest_float	[0.0, 0.4]	Probabilidad de descarte en las capas de atención y redes lineales intermedias ³⁰ .
activation	suggest_categorical	["gelu", "relu"]	Función de activación aplicada

	I		en la subcapa lineal intermedia del bloque feedforward ³⁰ .
learning_rate	suggest_float	[1e-5, 1e-3] (Log)	Tasa de aprendizaje. Los Transformers son muy sensibles a variaciones altas, requiriendo sintonización fina.

PatchTST

El modelo PatchTST replantea el procesamiento secuencial en Transformers mediante dos principios básicos: la segmentación de la serie en parches contiguos (*patching*) y la independencia de canales (*channel-independence*)²⁹. El parcheo consiste en agrupar pasos de tiempo cercanos en subseries discretas (parches) que funcionan como tokens de entrada, lo que permite capturar información semántica local de forma natural³². Por su parte, la independencia de canales separa las variables de una serie multivariada y las procesa como secuencias univariadas individuales que comparten los pesos de la red de atención, eliminando el riesgo de que el modelo aprenda correlaciones espurias entre canales diferentes²⁹.

Dado un horizonte temporal de entrada de longitud L , un parche de longitud P y un paso de deslizamiento S , la serie se transforma en un número de tokens N gobernado por la siguiente relación matemática³³:

$$N \approx \left\lfloor \frac{L - P}{S} \right\rfloor + 1$$

Esto disminuye la complejidad del cálculo de autoatención de $\mathcal{O}(L^2)$ a una escala cuadrática de parches $\mathcal{O}(N^2)$ ³². En sistemas no estacionarios, la integración de la Normalización de Instancia Reversible (RevIn) y sus parámetros afines actúan como hiperparámetros cruciales en el espacio de búsqueda de Optuna para evitar que el modelo aprenda tendencias inestables²⁹.

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
patch_len	suggest_int	[4, 32] (Paso: 4)	Dimensión temporal P de la ventana local agrupada para formar un token semántico ²⁸ .
stride	suggest_int	[2, 16] (Paso: 2)	Magnitud del paso S para el desplazamiento del parcheador sobre el eje temporal ²⁸ .
encoder_layers	suggest_int	[2, 6]	Cantidad de bloques de codificación Transformer paralelos independientes de canal ²⁹ .
n_heads	suggest_categorical	[4, 8, 16]	Cabezales de atención múltiple encargados de procesar la matriz de parches ²⁹ .
hidden_size	suggest_int	[64, 256] (Paso: 64)	Dimensión interna proyectada de la representación de parches y embeddings ²⁹ .
linear_hidden_size	suggest_int	[128, 512] (Paso: 64)	Dimensión de las capas lineales en la etapa de

			proyección de la cabeza del modelo ²⁹ .
dropout	suggest_float	[0.0, 0.4]	Regularización por descarte aplicada en las conexiones residuales del codificador ²⁹ .
attn_dropout	suggest_float	[0.0, 0.3]	Tasa de descarte aleatorio específica para suavizar la distribución de pesos de atención ²⁹ .
revin	suggest_categorica 	[True, False]	Activa o desactiva la normalización reversible para manejar derivas estadísticas ²⁹ .

xLSTM (Extended LSTM)

La arquitectura de memoria extendida xLSTM supera las limitaciones históricas de escala y la incapacidad de actualizar valores almacenados que aquejaban a las redes recurrentes clásicas frente a los Transformers, logrando escalar de manera eficiente a miles de millones de parámetros⁶. xLSTM introduce dos variantes estructurales integradas en bloques residuales¹¹: la variante sLSTM, que mantiene una memoria escalar secuencial con mezcla de estados recurrentes pero incorpora compuertas exponenciales dinámicas combinadas con un estabilizador logarítmico para mitigar desbordamientos numéricos⁶, y la variante mLSTM, que elimina la mezcla de memoria recurrente para lograr paralelización total en hardware moderno, utilizando una memoria matricial de alta capacidad regulada por una regla de actualización de covarianza de tipo producto externo⁶.

El balance dinámico del sLSTM se regula mediante el estado normalizador n_t y el término estabilizador m_t que pondera la magnitud de las compuertas exponenciales¹¹:

$$\begin{aligned}
m_t &= \max(\log f_t + m_{t-1}, \log i_t) \\
i'_t &= \exp(\log i_t - m_t) \\
f'_t &= \exp(\log f_t + m_{t-1} - m_t) \\
c_t &= f'_t c_{t-1} + i'_t z_t \\
n_t &= f'_t n_{t-1} + i'_t
\end{aligned}$$

$$\tilde{h}_t = \frac{c_t}{n_t}$$

Por su parte, el módulo mLSTM requiere proyectar matrices de memoria de dimensiones $C_t \in \mathbb{R}^{d_{qk} \times d_{hv}}$, lo que exige sintonizar parámetros específicos como el tamaño de bloque de la proyección lineal (qkv_proj_blocksize) y los tamaños de los filtros convolucionales unidimensionales (conv1d_kernel_size) internos que preceden a los mapeos lógicos¹¹.

Parámetro	Tipo de Sugerencia (Optuna)	Espacio de Búsqueda Recomendado	Justificación Técnica e Impacto del Parámetro
xlstm_type	suggest_categorical	["slstm", "mlstm"]	slstm prioriza el seguimiento de estados continuos mediante mezcla de memoria; mlstm prioriza paralelización masiva y capacidad matricial ³⁶ .
hidden_size	suggest_int	[32, 256] (Paso: 32)	Dimensión del vector latente que representa los estados ocultos y de memoria ⁴¹ .
num_layers	suggest_int	[1, 4]	Cantidad de bloques

			recurrentes xLSTM apilados con conexiones de tipo residual ³⁶ .
conv1d_kernel_size	suggest_categorica l	[2, 4, 6]	Tamaño del kernel convolucional de filtrado previo de secuencias en cada bloque xLSTM ¹¹ .
p_factor	suggest_float	[1.0, 2.0] (Paso: 0.1)	Factor multiplicador de proyección de la red de alimentación totalmente conectada interna ¹¹ .
dropout	suggest_float	[0.0, 0.4]	Probabilidad de descarte aplicada en el canal de salida de los bloques residuales ⁴¹ .
qkv_proj_blocksize	suggest_categorica l	[2, 4, 8]	Dimensión del bloque para proyecciones matriciales de atención Query-Key-Value en mLSTM ¹¹ .
num_heads	suggest_categorica l	[2, 4, 8]	Número de cabezales independientes de distribución de memoria paralela en mLSTM ¹¹ .

Implementación en Python de la Función Objetivo de Optuna

El siguiente bloque de código proporciona una plantilla estructurada y modular para inicializar

el espacio de búsqueda unificado en Optuna, aislando los parámetros de cada modelo bajo una estructura condicional dinámica y respetando las restricciones de divisibilidad internas de los cabezales de atención y variantes estructurales.

Python

```
import optuna
import torch
```

```
def sample_random_forest(trial):
    """
    Construye el espacio de búsqueda probabilístico para el modelo RandomForest.
    """
    return {
        "model_type": "RandomForest",
        "n_estimators": trial.suggest_int("rf_n_estimators", 50, 1000, step=50),
        "max_depth": trial.suggest_categorical(
            "rf_max_depth", [None, 5, 10, 20, 30, 50]
        ),
        "min_samples_split": trial.suggest_int("rf_min_samples_split", 2, 20),
        "min_samples_leaf": trial.suggest_int("rf_min_samples_leaf", 1, 20),
        "max_features": trial.suggest_categorical(
            "rf_max_features", ["sqrt", "log2", None]
        ),
        "bootstrap": trial.suggest_categorical("rf_bootstrap", [True, False]),
        "criterion": trial.suggest_categorical("rf_criterion", ["gini", "entropy"])
    }
```

```
def sample_lstm(trial):
    """
    Especifica los parámetros de sintonización para el modelo recurrente LSTM.
    """
    return {
        "model_type": "LSTM",
        "hidden_size": trial.suggest_int("lstm_hidden_size", 16, 512, step=16),
        "num_layers": trial.suggest_int("lstm_num_layers", 1, 4),
        "dropout": trial.suggest_float("lstm_dropout", 0.0, 0.5)
```

```

"learning_rate" trial.suggest_float("lstm_lr" 1e-5 1e-2 log=True
"batch_size" trial.suggest_categorical(
    "lstm_batch_size" 16 32 64 128 256
)
"optimizer" trial.suggest_categorical(
    "lstm_optimizer" "Adam" "AdamW" "RMSprop" "SGD"
)

```

```

def sample_gru(trial):
    """
    Configura el espacio de búsqueda optimizado para el bloque recurrente GRU.
    """
    return
    "model_type" "GRU"
    "hidden_size" trial.suggest_int("gru_hidden_size" 16 512 step=16
    "num_layers" trial.suggest_int("gru_num_layers" 1 4
    "dropout" trial.suggest_float("gru_dropout" 0.0 0.5
    "learning_rate" trial.suggest_float("gru_lr" 1e-5 1e-2 log=True
    "weight_decay" trial.suggest_float("gru_weight_decay" 1e-6 1e-2 log=True
    "batch_size" trial.suggest_categorical(
        "gru_batch_size" 16 32 64 128 256
    )

```

```

def sample_cnn_lstm(trial):
    """
    Gestiona el espacio de hiperparámetros híbridos convolucionales y secuenciales.
    """
    return
    "model_type" "CNN-LSTM"
    "conv_filters" trial.suggest_int("cnn_conv_filters" 16 128 step=16
    "kernel_size" trial.suggest_int("cnn_kernel_size" 2 8
    "pool_size" trial.suggest_int("cnn_pool_size" 2 4
    "lstm_hidden_size" trial.suggest_int("cnn_lstm_hidden_size" 32 256 step=32
    "lstm_layers" trial.suggest_int("cnn_lstm_layers" 1 3
    "dropout" trial.suggest_float("cnn_dropout" 0.1 0.5
    "learning_rate" trial.suggest_float("cnn_lr" 1e-5 1e-2 log=True

```

```

def sample_tcn(trial):
    """
    Establece los parámetros que gobiernan el campo receptivo convolucional de la TCN.
    """
    channels_options =
        16 32 64
        32 64 128
        64 128 256

    # Muestreo de las configuraciones de lista de canales
    num_channels_idx = trial.suggest_categorical("tcn_channels_idx" 0 1 2
    num_channels = channels_options[num_channels_idx]

    return
    "model_type" "TCN"
    "depth" trial.suggest_int("tcn_depth" 2 6
    "num_channels" num_channels
    "kernel_size" trial.suggest_int("tcn_kernel_size" 2 8
    "dilation_reset" trial.suggest_categorical
        "tcn_dilation_reset" None 8 16 32
    )
    "dropout" trial.suggest_float("tcn_dropout" 0.0 0.5
    "use_norm" trial.suggest_categorical
        "tcn_use_norm" "weight_norm" "batch_norm" "none"
    )
    "use_skip_connections" trial.suggest_categorical
        "tcn_use_skip" True False
    )

```

```

def sample_transformer(trial):
    """
    Construye las proyecciones lineales garantizando que d_model sea divisible por nhead.
    """
    nhead = trial.suggest_categorical("trans_nhead" 2 4 8 16
    # Factor entero para resguardar la divisibilidad analítica de d_model
    d_model_multiplier = trial.suggest_int("trans_d_model_multiplier" 8 32
    d_model = nhead * d_model_multiplier

    return
    "model_type" "Time_Series_Transformer"
    "nhead" nhead

```

```

        "d_model" d_model
        "num_layers" trial.suggest_int("trans_num_layers" 1 6)
        "dim_feedforward" trial.suggest_int(
            "trans_dim_feedforward" 64 1024 step=64
        )
        "dropout" trial.suggest_float("trans_dropout" 0.0 0.4)
        "activation" trial.suggest_categorical("trans_activation" ["gelu", "relu"])
        "learning_rate" trial.suggest_float("trans_lr" 1e-5 1e-3 log=True)
    
```

```
def sample_patch_tst(trial):
```

```

    """
    Formula el espacio de sintonización para el modelo PatchTST.
    """
    return
    "model_type" "PatchTST"
    "patch_len" trial.suggest_int("patch_len" 4 32 step=4)
    "stride" trial.suggest_int("patch_stride" 2 16 step=2)
    "encoder_layers" trial.suggest_int("patch_encoder_layers" 2 6)
    "n_heads" trial.suggest_categorical("patch_n_heads" [4, 8, 16])
    "hidden_size" trial.suggest_int("patch_hidden_size" 64 256 step=64)
    "linear_hidden_size" trial.suggest_int(
        "patch_linear_hidden_size" 128 512 step=64
    )
    "dropout" trial.suggest_float("patch_dropout" 0.0 0.4)
    "attn_dropout" trial.suggest_float("patch_attn_dropout" 0.0 0.3)
    "revin" trial.suggest_categorical("patch_revin" [True, False])

```

```
def sample_xlstm(trial):
```

```

    """
    Resuelve la selección condicional entre bloques sLSTM y mLSTM en el xLSTM.
    """
    xlstm_type = trial.suggest_categorical("xlstm_type" ["slstm", "mlstm"])

    config =
        "model_type" "xLSTM"
        "xlstm_type" xlstm_type
        "hidden_size" trial.suggest_int("xlstm_hidden_size" 32 256 step=32)
        "num_layers" trial.suggest_int("xlstm_num_layers" 1 4)
        "conv1d_kernel_size" trial.suggest_categorical(

```

```

        "xlstm_conv_kernel" 2 4 6
    )

    "p_factor" trial.suggest_float("xlstm_p_factor" 1.0 2.0 step=0.1)
    "dropout" trial.suggest_float("xlstm_dropout" 0.0 0.4)

    if xlstm_type == "mlstm":
        # Parámetros exclusivos del módulo mLSTM de tipo matricial
        config["qkv_proj_blocksize"] = trial.suggest_categorical(
            "xlstm_qkv_block" 2 4 8
        )

        config["num_heads"] = trial.suggest_categorical("xlstm_num_heads" 2 4 8)

    return config

def objective(trial):
    """
    Función objetivo integrada para la búsqueda jerárquica y condicional en Optuna.
    """
    # Selección dinámica de la familia de modelos a optimizar
    model_family = trial.suggest_categorical(
        "model_family"
        "RandomForest"
        "LSTM"
        "GRU"
        "CNN-LSTM"
        "TCN"
        "Transformer"
        "PatchTST"
        "xLSTM"
    )

    if model_family == "RandomForest":
        hyperparameters = sample_random_forest(trial)
    elif model_family == "LSTM":
        hyperparameters = sample_lstm(trial)
    elif model_family == "GRU":
        hyperparameters = sample_gru(trial)
    elif model_family == "CNN-LSTM":
        hyperparameters = sample_cnn_lstm(trial)

```

```

elif model_family == "TCN":
    hyperparameters = sample_tcn(trial)
elif model_family == "Transformer":
    hyperparameters = sample_transformer(trial)
elif model_family == "PatchTST":
    hyperparameters = sample_patch_tst(trial)
elif model_family == "xLSTM":
    hyperparameters = sample_xlstm(trial)
else:
    raise ValueError("Familia de modelos desconocida.")

# Simulación de un valor de pérdida devuelto tras el ciclo de entrenamiento.
# En un entorno real, se inicializaría el modelo utilizando el diccionario 'hyperparameters',
# se entrenaría con el conjunto de entrenamiento correspondiente y se devolvería la
# métrica
# de pérdida en el conjunto de validación (p. ej., el error cuadrático medio, MSE).
dummy_val_loss = 0.05
return dummy_val_loss

if __name__ == "__main__":
    # Inicialización del estudio utilizando el optimizador TPE adaptativo
    study = optuna.create_study(
        direction="minimize", sampler=optuna.samplers.TPESampler(seed=42)
    )

    study.optimize(objective, n_trials=100)

    print("Hiperparámetros de la mejor ejecución:")
    print(study.best_params)

```

Análisis Técnico Compartido de las Dinámicas de Ajuste

La sintonización coordinada de hiperparámetros mediante optimización bayesiana revela patrones de co-dependencia y comportamientos estructurales bien diferenciados entre las distintas familias de modelos analizadas²:

- Sensibilidad Paramétrica y Regularización:** Los modelos de aprendizaje profundo basados en mecanismos de atención (como Transformers y PatchTST) exhiben una alta sensibilidad a la tasa de aprendizaje (`learning_rate`) y a los esquemas de inicialización de pesos³⁴. Mientras que un bosque aleatorio tolera rangos amplios de hiperparámetros sin divergir numéricamente, un Transformer puede colapsar a predicciones de valor medio si la tasa de aprendizaje es excesiva o si la regularización no amortigua la dispersión de

los pesos de atención³⁰. Por ello, las búsquedas con Optuna en modelos basados en atención se ven favorecidas por el uso de distribuciones logarítmicas de grano fino sobre la tasa de aprendizaje².

- **Sistemas de Normalización Estacionaria:** En el modelado de series de tiempo no estacionarias, la inclusión de normalizaciones dinámicas como RevIn (dentro de PatchTST y adaptaciones afines) altera radicalmente el espacio de búsqueda²⁹. Al remover de manera reversible las tendencias de variación del valor medio y la varianza a lo largo de las secuencias, estas capas aíslan el aprendizaje del desfase estacional, lo que permite al optimizador bayesiano concentrarse en sintonizar dimensiones de red más eficientes y ligeras sin riesgo de inestabilidad matemática²⁹.
- **Compromiso entre Paralelización e Inferencia Secuencial:** El espacio de búsqueda de la arquitectura xLSTM pone de manifiesto el compromiso existente entre la representatividad matemática y el coste computacional del entrenamiento⁶. Al elegir entre slstm (con mezcla secuencial de memoria exponencial y estabilización de normalización) y mlstm (con paralelización libre de recurrencia mediante matrices de covarianza), el optimizador bayesiano no solo selecciona un conjunto de hiperparámetros de escala, sino que determina también la velocidad de procesamiento de los lotes sobre el hardware⁶. En este escenario, mLSTM destaca por su gran capacidad para escalar con la longitud de secuencia, ya que se beneficia directamente de una computación en paralelo altamente eficiente, similar a la de los cabezales de autoatención de los Transformers tradicionales¹¹.

Obras citadas

1. Hyperparameter Tuning: A Practical Guide to Optimizing ML Models - Zerve, <https://www.zerve.ai/blog/optimizing-performance-a-hands-on-guide-to-hyperparameter-tuning>
2. Two-stage random search–Bayesian optimization for CNN- based short-term load forecasting - Bulletin of Electrical Engineering and Informatics, <http://www.beei.org/index.php/EEI/article/download/11030/4809>
3. RandomForest+Optuna - Kaggle, <https://www.kaggle.com/code/aaronisomaisom3/randomforest-optuna>
4. How Many Trees in a Random Forest? A Revisited Approach With Plateau Search and Optuna Integration - arXiv, <https://arxiv.org/html/2606.03549v2>
5. Hyperparameter Tuning with Optuna for Data Science in Microsoft Fabric, <https://community.fabric.microsoft.com/t5/Data-Science-Community-Blog/Hyperparameter-Tuning-with-Optuna-for-Data-Science-in-Microsoft/ba-p/4810216>
6. xLSTM: Extended Long Short Term Memory | by Nagesh Singh Chauhan - GoPenAI, <https://blog.gopenai.com/xlstm-extended-long-short-term-memory-129ce6d4d741>
7. A Hybrid 1D-CNN–LSTM–MHA Model for Short-Term Probabilistic Photovoltaic Power Forecasting - MDPI, <https://www.mdpi.com/2673-9941/6/3/30>

8. Automatic selection of the best neural architecture for time series forecasting via multi-objective optimization and Pareto optimality conditions - arXiv, <https://arxiv.org/html/2501.12215v1>
9. Gated Recurrent Unit (GRU) for Forecasting Hourly Energy Fluctuations - Universitas Negeri Malang, <https://journal2.um.ac.id/index.php/fespe/article/download/51438/12904>
10. A hyperparameter-space clustering methodology of residential electricity loads - ENPOWER, <https://www.enpower-project.eu/sites/default/files/2025-12/A%20hyperparameter-space%20clustering%20methodology%20of%20residential%20electricity%20loads.pdf>
11. Extended LSTM to Enhance Learner Performance Prediction - MDPI, <https://www.mdpi.com/1999-4893/19/4/251>
12. xLSTM: Extended Long Short-Term Memory - OpenReview, <https://openreview.net/pdf?id=ARAxPPIAhq>
13. Application of Hybrid CNN-LSTM Architecture with Optuna Optimization for Weather Image Captioning, <https://ejournal.seaninstitute.or.id/index.php/esaprom/article/download/8119/6208>
14. LSTM Network Hyperparameter Optimization for Stock Price Prediction Using the Optuna Framework, <https://eprints.uad.ac.id/41357/1/3-LSTM%20Network%20Hyperparameter%20Optimization%20for%20Stock%20Price%20Prediction%20Using%20the%20Optuna%20Framework.pdf>
15. Time-Series-Transformer-Pytorch/transformer_model.ipynb at main - GitHub, https://github.com/ctxj/Time-Series-Transformer-Pytorch/blob/main/transformer_model.ipynb
16. (PDF) Transformers Outperform Traditional Forecasting Models and Perform Comparably to Recurrent Neural Networks in the Prediction of Emergency Department Visits using Calendar and Meteorological Data - ResearchGate, https://www.researchgate.net/publication/400867014_Transformers_Outperform_Traditional_Forecasting_Models_and_Perform_Comparably_to_Recurrent_Neural_Networks_in_the_Prediction_of_Emergency_Department_Visits_using_Calendar_and_Meteorological_Data
17. Deep learning Calabi-Yau four folds with hybrid and recurrent neural network architectures - CERN, https://scoap3-prod-backend.s3.cern.ch/media/harvested_files/10.1016/j.nuclphysb.2025.116832/main.pdf
18. pytorch-tcn - PyPI, <https://pypi.org/project/pytorch-tcn/1.1.0/>
19. EFFECTIVENESS OF CNN-LSTM MODELS USED FOR APPLE STOCK FORECASTING - CSUSB ScholarWorks, <https://scholarworks.lib.csusb.edu/cgi/viewcontent.cgi?article=3143&context=etd>
20. Simplified Temporal Convolutional-based Channel Estimation for a WiFi Vehicular Communication Channel - arXiv, <https://arxiv.org/html/2606.10511v1>
21. Understanding Temporal Convolutional Networks (TCNs) — From CNN Basics to Full Sequence Mastery | by Rehan | Medium,

- <https://medium.com/@rehan020345/understanding-temporal-convolutional-nets-works-tcns-from-cnn-basics-to-full-sequence-mastery-57d0804ad8c8>
22. pytorch-tcn - PyPI, <https://pypi.org/project/pytorch-tcn/1.0.0/>
 23. Tensorflow eager implementation of Temporal Convolutional Network (TCN) - GitHub, <https://github.com/Baichenjia/Tensorflow-TCN>
 24. pyhealth.models.TCN - Read the Docs, <https://pyhealth.readthedocs.io/en/latest/api/models/pyhealth.models.TCN.html>
 25. TCN-XGBoost Hybrid Model for Daily FCR Prediction and One Step Forecasting in Broiler Farm Salman Alfarisi Rizwana1, Wiwin Sry A, <https://journals.ap2.pt/index.php/ais/article/download/1017/670/1756>
 26. A Temporal Convolutional Network–Bidirectional Long Short-Term Memory (TCN–BiLSTM) Prediction Model for Temporal Faults in Industrial Equipment - MDPI, <https://www.mdpi.com/2076-3417/15/4/1702>
 27. Minimal Time Series Transformer. The original “Attention Is All You...” | by Joni Kamarainen | Medium, <https://medium.com/@jkamarai/minimal-time-series-transformer-2143ec7be509>
 28. notebook8d69b0ef0c - Kaggle, <https://www.kaggle.com/code/sandragirgis/notebook8d69b0ef0c>
 29. PatchTST - Nixtla - Nixtlaverse, <https://nixtlaverse.nixtla.io/neuralforecast/models.patchtst.html>
 30. PatchTST - Hugging Face, https://huggingface.co/docs/transformers/model_doc/patchtst
 31. Transformer Forecasting with PyTorch | Data-Driven Engineering - APMonitor, <https://apmonitor.com/dde/index.php/Main/LLMTransformers>
 32. [PatchTST] A Time Series is Worth 64 Words: Long-Term Forecasting with Transformers, <https://letter-night.tistory.com/450>
 33. A Time Series is Worth 64 Words: Long-term Forecasting with Transformers - ar5iv - arXiv, <https://ar5iv.labs.arxiv.org/html/2211.14730>
 34. Lithium-ion battery RUL prediction based on optimized VMD-SSA-PatchTST algorithm, <https://pmc.ncbi.nlm.nih.gov/articles/PMC12287278/>
 35. Time Series Forecasting with Transformers: From Theory to Implementation - Miraflow AI, <https://miraflow.ai/blog/time-series-forecasting-transformers-theory-to-implementation>
 36. xLSTM: Extended Long Short-Term Memory - OpenReview, [https://openreview.net/forum?id=ARAXPPIAhq&referrer=%5Bthe%20profile%20of%20Michael%20K%20Kopp%5D\(%2Fprofile%3Fid%3D~Michael_K_Kopp1\)](https://openreview.net/forum?id=ARAXPPIAhq&referrer=%5Bthe%20profile%20of%20Michael%20K%20Kopp%5D(%2Fprofile%3Fid%3D~Michael_K_Kopp1))
 37. xLSTM: Recurrent Neural Network Architectures for Scalable and Efficient Large Language Models - Maximilian Beck, https://maxbeck.ai/resources/phd_thesis_beck_xlstm.pdf
 38. xLSTM 7B: A Recurrent LLM for Fast and Efficient Inference - GitHub, <https://raw.githubusercontent.com/mlresearch/v267/main/assets/beck25b/beck25b.pdf>
 39. mLSTM-4classes - Kaggle, <https://www.kaggle.com/code/swaroopsrisailam/mlstm-4classes>

40. NX-AI/xlstm: Official repository of the xLSTM. · GitHub, <https://github.com/nx-ai/xlstm>
41. xLSTMTime — pytorch-forecasting documentation - Read the Docs, https://pytorch-forecasting.readthedocs.io/en/stable/api/pytorch_forecasting.models.xlstm._xlstm.xLSTMTime.html
42. myscience/x-lstm: Pytorch implementation of the xLSTM model by Beck et al. (2024) - GitHub, <https://github.com/myscience/x-lstm>
43. A Comparative Study of Day-Ahead Wind Power Forecasting Models for a Single Wind Farm Under Strict Chronological Splitting and Unified Hyperparameter Tuning Conditions - MDPI, <https://www.mdpi.com/1996-1073/19/12/2784>